Experiment 10:TSP using Genetic Algorithm (GA)

Objective:

implementing the Traveling Salesman Problem (TSP) using genetic algorithms, plotting correlation plots on a dataset, visualizing relationships among data

Theory

The Traveling Salesman Problem (TSP) is a well-known combinatorial optimization problem in which a salesman must visit a set of cities exactly once and return to the starting city while minimizing the total travel distance. It is classified as an NP-hard problem due to its computational complexity, meaning that the time required to solve it increases exponentially with the number of cities.

A Genetic Algorithm (GA) is an optimization technique inspired by the principles of natural selection and genetics. It works with a population of possible solutions and improves them over successive generations using evolutionary operations. Each solution is evaluated using a fitness function, and better solutions are more likely to be selected for reproduction.

Key Components of Genetic Algorithm

1. Selection

Selection is the process of choosing the best individuals from the current population to serve as parents for the next generation. The selection is based on the fitness value of each individual. In the case of TSP, fitness is usually defined as the inverse of the total distance of the route. Individuals with shorter routes have higher fitness and are more likely to be selected. This ensures that better solutions are carried forward to the next generation.

2. Crossover

Crossover is a genetic operator used to combine two parent solutions to generate new offspring. It involves exchanging segments of the parent chromosomes to create a new route. In the TSP, special crossover techniques such as order crossover are used to ensure that each city appears exactly once in the offspring. This helps in preserving valid solutions while combining useful traits from both parents.

3. Mutation

Mutation introduces small random changes in a solution to maintain diversity within the population. It prevents the algorithm from getting stuck in local optimal solutions. In the TSP, mutation is typically performed by swapping two cities in the route. This creates a new variation of the solution and allows the algorithm to explore different possible routes.

```python
# ============================================================
# EXPERIMENT:
# TSP using Genetic Algorithm + Data Visualization
# ============================================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import random

# ============================================================
# PART A: TSP USING GENETIC ALGORITHM
# ============================================================

# Fixed city coordinates
cities = np.array([
    [0.92, 0.43],  # 0
    [1.00, 0.97],  # 1
    [0.85, 0.30],  # 2
    [0.38, 0.85],  # 3
    [0.30, 0.16],  # 4
    [0.55, 0.93],  # 5
    [0.70, 0.58],  # 6
    [0.12, 0.61],  # 7
    [0.98, 0.14],  # 8
    [0.52, 0.88]   # 9
])

num_cities = len(cities)

# -------------------------
# Distance Function
# -------------------------
def calculate_distance(route):
    total_distance = 0
    for i in range(len(route)):
        x1, y1 = cities[route[i]]
        x2, y2 = cities[route[(i + 1) % len(route)]]
```

```
        total_distance += np.linalg.norm([x2 - x1, y2 - y1])
    return total_distance

# -------------------------
# Initial Population
# -------------------------
def create_population(size):
    return [random.sample(range(num_cities), num_cities) for _ in
range(size)]

# -------------------------
# Fitness Function
# -------------------------
def fitness(route):
    return 1 / calculate_distance(route)

# -------------------------
# Selection
# -------------------------
def selection(population):
    population = sorted(population, key=lambda x: fitness(x),
reverse=True)
    return population[:len(population)//2]

# -------------------------
# Crossover (Order Based)
# -------------------------
def crossover(parent1, parent2):
    start, end = sorted(random.sample(range(num_cities), 2))
    child = [-1] * num_cities

    # Copy segment from parent1
    child[start:end] = parent1[start:end]

    # Fill remaining from parent2
    pointer = 0
    for city in parent2:
        if city not in child:
            while child[pointer] != -1:
                pointer += 1
            child[pointer] = city

    return child

# -------------------------
# Mutation (Swap)
# -------------------------
def mutation(route, mutation_rate=0.02):
```

```
    if random.random() < mutation_rate:
        i, j = random.sample(range(num_cities), 2)
        route[i], route[j] = route[j], route[i]
    return route

# -------------------------
# Genetic Algorithm
# -------------------------
def genetic_algorithm(generations=200, pop_size=100):
    population = create_population(pop_size)
    best_distances = []

    for gen in range(generations):

        # Selection
        population = selection(population)

        # New generation
        new_population = []
        while len(new_population) < pop_size:
            parent1, parent2 = random.sample(population, 2)
            child = crossover(parent1, parent2)
            child = mutation(child)
            new_population.append(child)

        population = new_population

        # Track best route
        best_route = min(population, key=lambda x:
calculate_distance(x))
        best_distances.append(calculate_distance(best_route))

    return best_route, best_distances

# -------------------------
# Run GA
# -------------------------
best_route, best_distances = genetic_algorithm()

print("\n===== TSP RESULT =====")
print("Best Route:", best_route)
print("Minimum Distance:", calculate_distance(best_route))

# ============================================================
# PLOT TSP ROUTE WITH DISTANCE LABELS
# ============================================================

route = best_route + [best_route[0]]
```

```python
plt.figure()

# Plot nodes
plt.scatter(cities[:, 0], cities[:, 1])

# Draw edges + distances
for i in range(len(route) - 1):
    x1, y1 = cities[route[i]]
    x2, y2 = cities[route[i + 1]]

    # Line
    plt.plot([x1, x2], [y1, y2])

    # Distance
    dist = np.linalg.norm([x2 - x1, y2 - y1])

    # Midpoint label
    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2
    plt.text(mid_x, mid_y, f"{dist:.2f}", fontsize=9)

# Label nodes
for i, (x, y) in enumerate(cities):
    plt.text(x, y, str(i), fontsize=12)

plt.title("Optimized TSP Route using Genetic Algorithm")
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid()
plt.show()

# ============================================================
# CONVERGENCE GRAPH
# ============================================================

plt.figure()
plt.plot(best_distances)
plt.title("GA Convergence")
plt.xlabel("Generation")
plt.ylabel("Distance")
plt.grid()
plt.show()

# ============================================================
# PART B: DATA VISUALIZATION (NO DATASET NEEDED)
# ============================================================
```

```
np.random.seed(42)

feature1 = np.random.randint(10, 100, 50)
feature2 = feature1 + np.random.randint(-10, 10, 50)
feature3 = np.random.randint(5, 80, 50)
feature4 = feature2 + np.random.randint(-15, 15, 50)

df = pd.DataFrame({
    'Feature1': feature1,
    'Feature2': feature2,
    'Feature3': feature3,
    'Feature4': feature4
})

print("\n===== DATASET PREVIEW =====")
print(df.head())

# -------------------------
# Correlation Heatmap
# -------------------------
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# -------------------------
# Pairplot
# -------------------------
sns.pairplot(df)
plt.show()

# -------------------------
# Scatter Plot
# -------------------------
plt.figure()
sns.scatterplot(x='Feature1', y='Feature2', data=df)
plt.title("Scatter Plot (Feature1 vs Feature2)")
plt.show()

# -------------------------
# Regression Plot
# -------------------------
plt.figure()
sns.regplot(x='Feature1', y='Feature2', data=df)
plt.title("Regression Plot")
plt.show()

# -------------------------
```

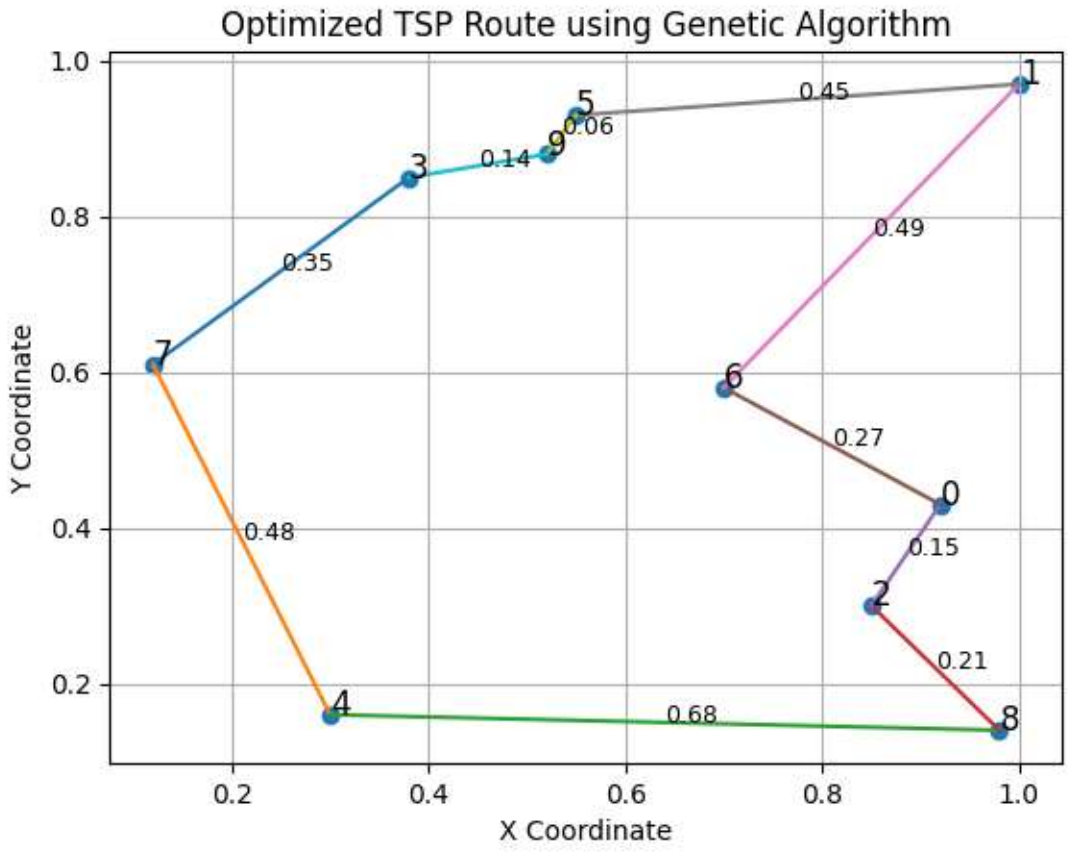
```
# Box Plot
# ------------------------
plt.figure()
sns.boxplot(data=df)
plt.title("Box Plot of Features")
plt.show()
# ============================================================
# END OF PROGRAM
# ============================================================
```

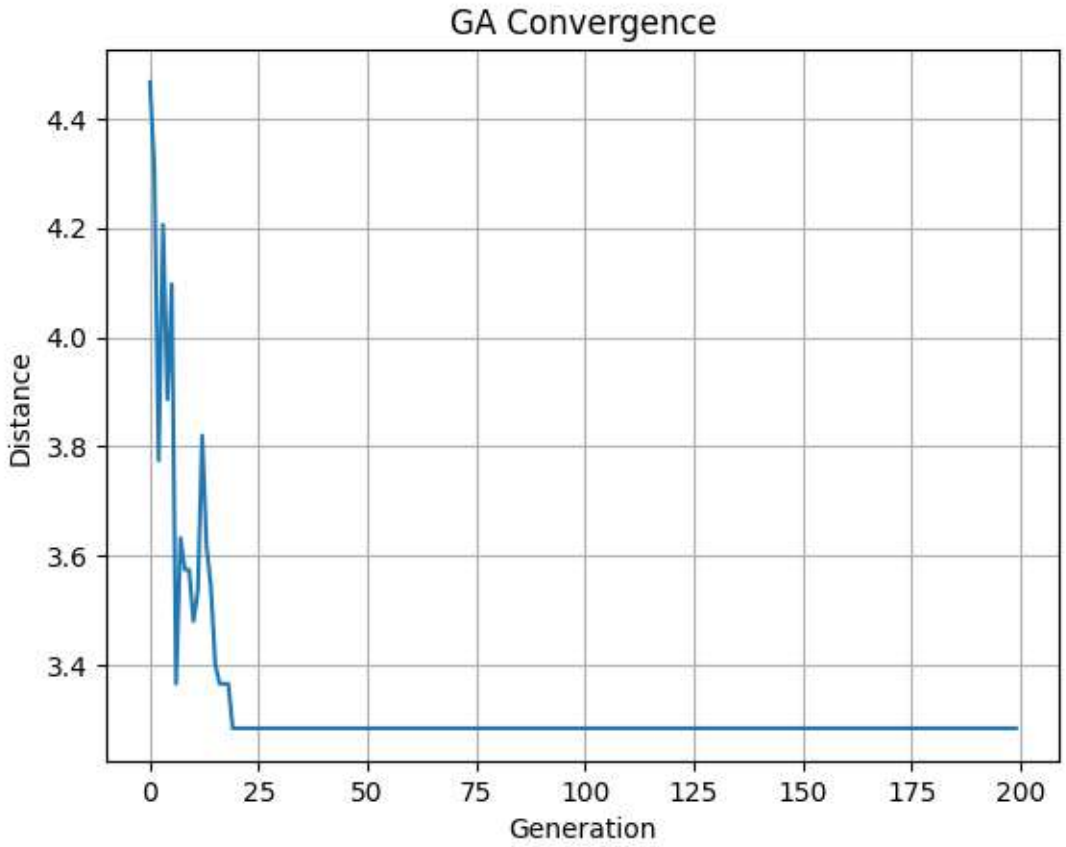
Output:

===== TSP RESULT =====

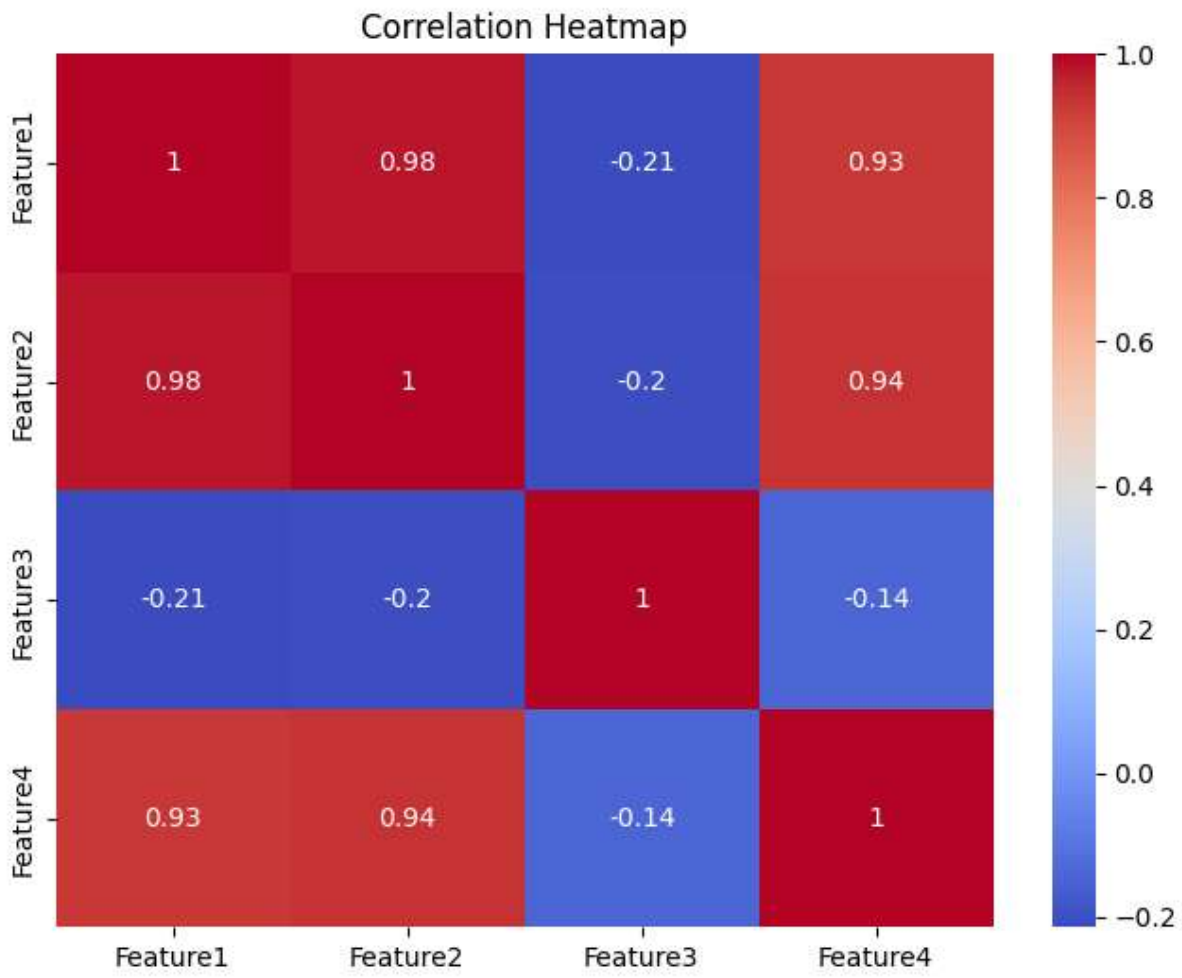
Best Route: [3, 7, 4, 8, 2, 0, 6, 1, 5, 9]

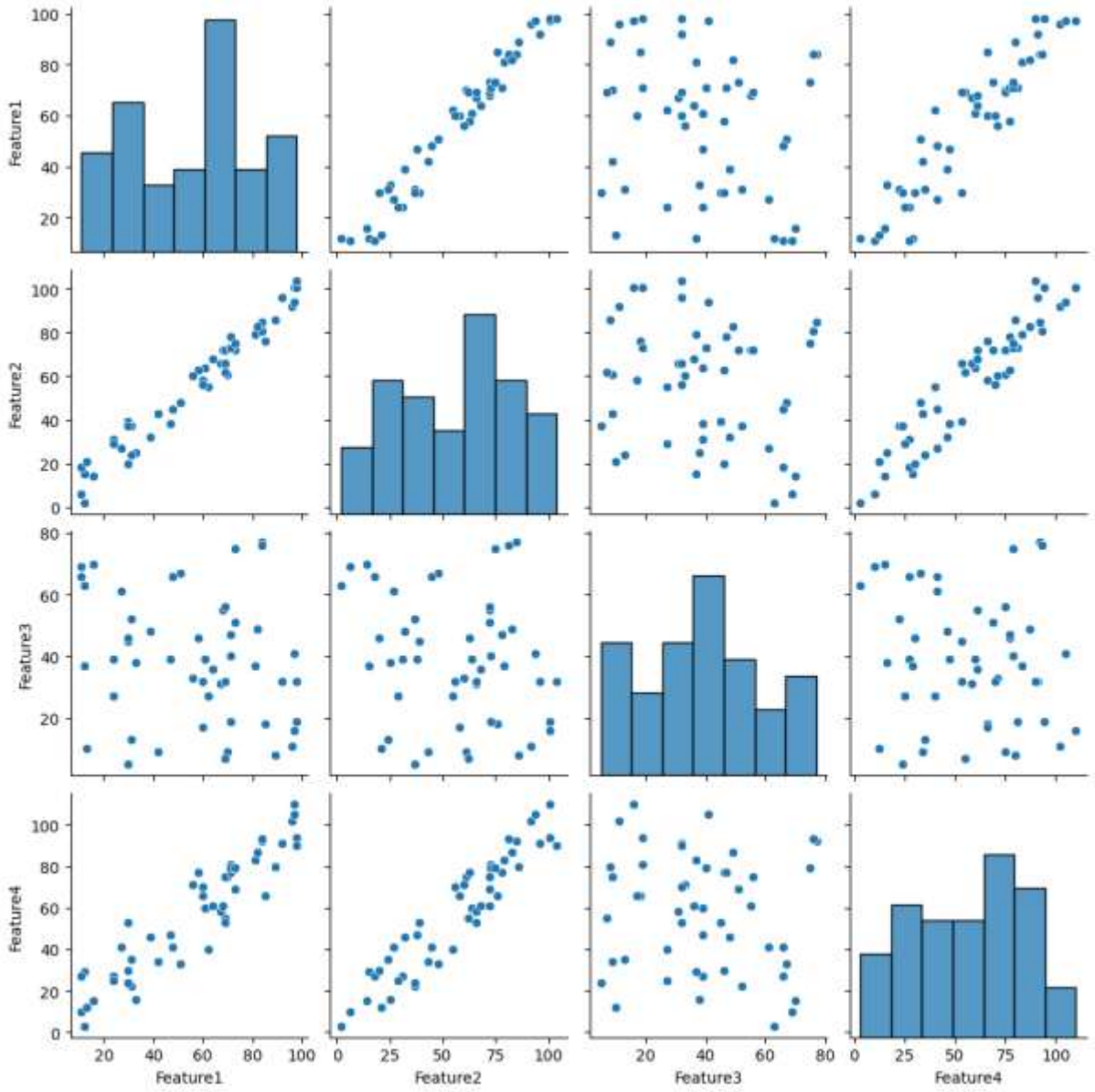
Minimum Distance: 3.2841676514760687

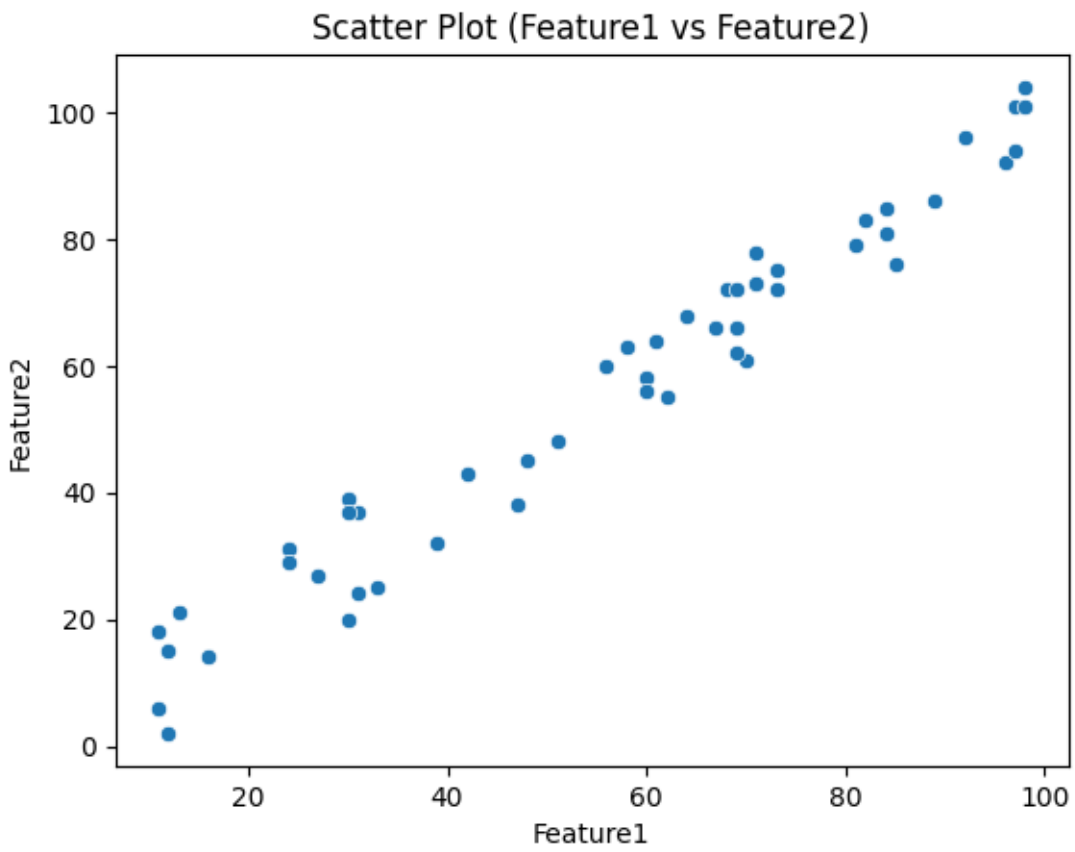


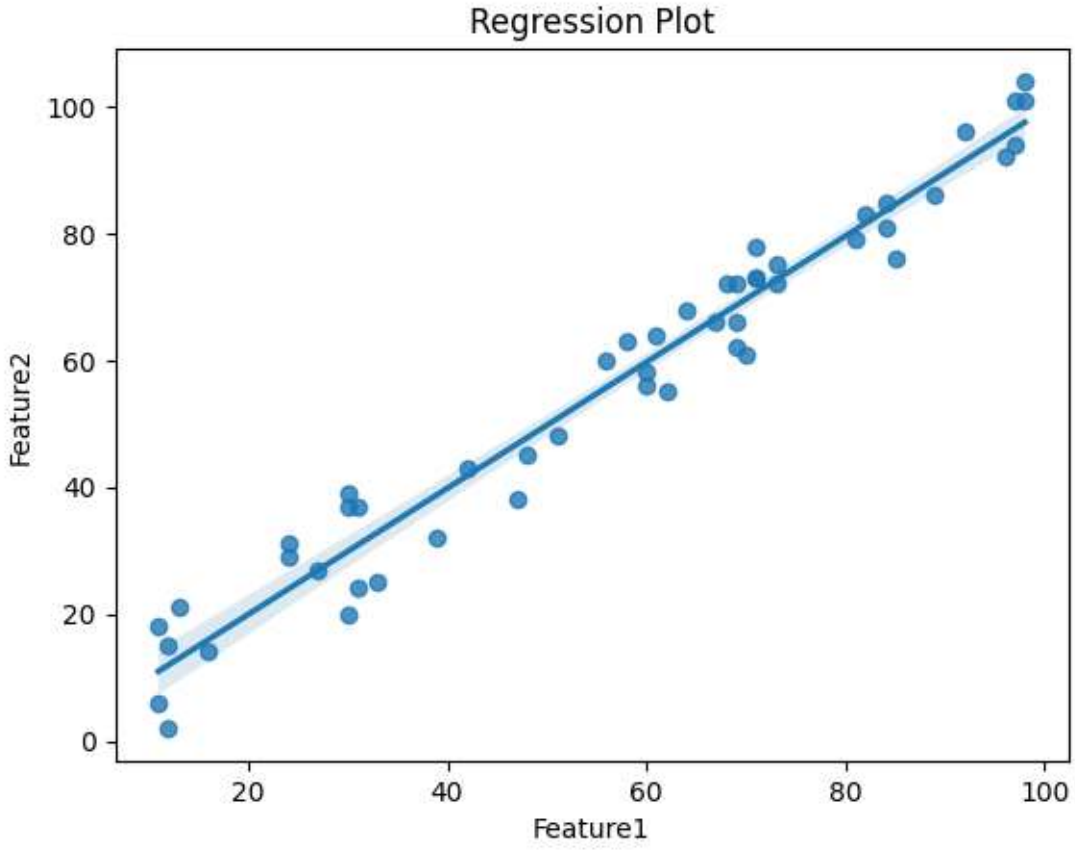


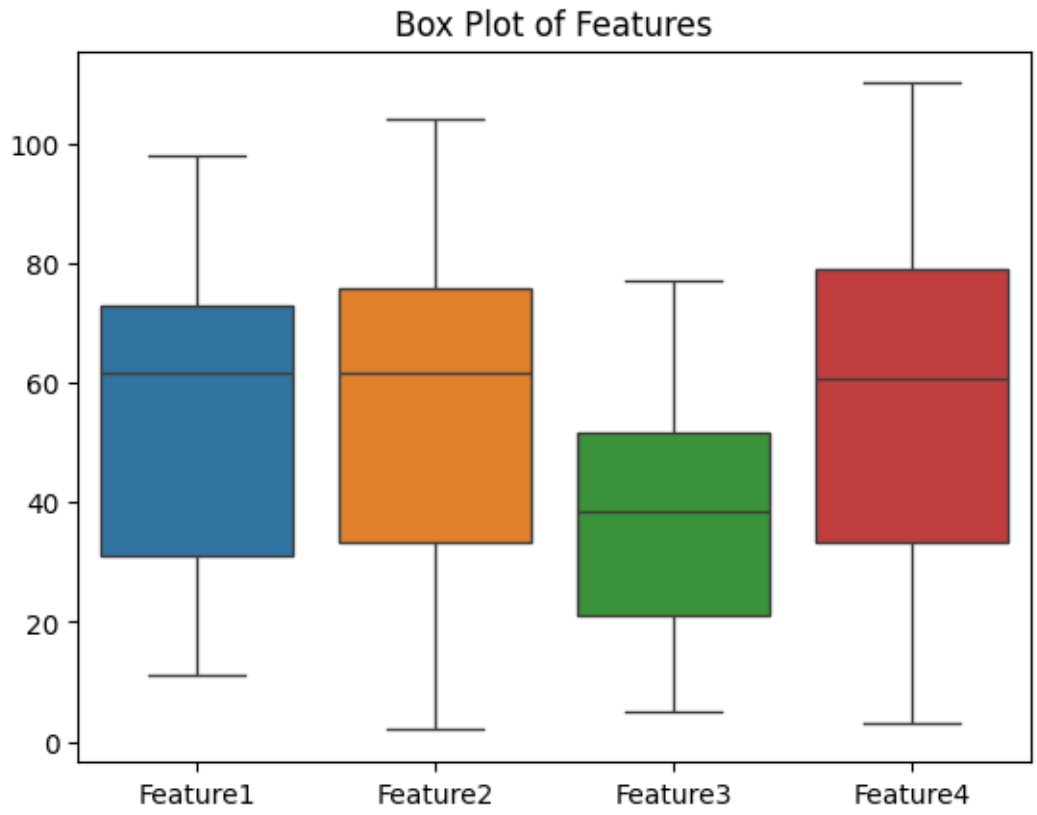

```
===== DATASET PREVIEW =====

   Feature1  Feature2  Feature3  Feature4

0        61        64        39        60

1        24        31        39        27

2        81        79        37        83

3        70        61         9        75

4        30        39        45        53
```


Correlation Heatmap
Feature1
Feature2
Feature3
Feature4
1
0.98
-0.21
0.93
0.98
1
-0.2
0.94
-0.21
-0.2
1
-0.14
0.93
0.94
-0.14
1
Feature1
Feature2
Feature3
Feature4
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Feature1
Feature2
Feature3
Feature4
Feature1
Feature2
Feature3
Feature4


Scatter Plot (Feature1 vs Feature2)
Feature2
Feature1
0
20
40
60
80
100
20
40
60
80
100


Regression Plot
Feature2
Feature1
0
20
40
60
80
100
20
40
60
80
100



Conclusion:

Successfully implemented.